

MQTT 协议快速入门

MQTT 协议快速入门 2025：基础知识和实用教程



2025-3-24

MQTT 教程

MQTT Protocol Explained

目录

- 什么是 MQTT?
- 为什么 MQTT 是适用于物联网的最佳协议?
- MQTT 的工作原理
- MQTT 的工作流程
- 开始使用 MQTT：快速教程
- 协议对比：MQTT 的核心优势
- MQTT 进阶
- 2025 年 MQTT 的 8 大趋势
- 深入学习 MQTT

在快速发展的物联网领域，高效的设备通信至关重要。MQTT 正是实现这一目标的核心协议。本指南将带您深入解析 MQTT 这一专为低带宽、高延迟网络设计的轻量级发布-订阅协议。我们将

系统讲解 MQTT 的基础原理、核心概念和实际应用场景，通过专业洞见与实操案例相结合，为您提供掌握 MQTT 协议、加速物联网项目的完整知识体系。

什么是 MQTT?

MQTT (Message Queuing Telemetry Transport) 是一种轻量级、基于发布-订阅模式的消息传输协议，适用于资源受限的设备和低带宽、高延迟或不稳定的网络环境。它在物联网应用中广受欢迎，能够实现传感器、执行器和其它设备之间的高效通信。

为什么 MQTT 是适用于物联网的最佳协议?

MQTT 所具有的适用于物联网特定需求的特点和功能，使其成为物联网领域最佳的协议之一。它的主要特点包括：

- **轻量级**：物联网设备通常在处理能力、内存和能耗方面受到限制。MQTT 开销低、报文小的特点使其非常适合这些设备，因为它消耗更少的资源，即使在有限的能力下也能实现高效的通信。
- **可靠**：物联网网络常常面临高延迟或连接不稳定的情况。MQTT 支持多种 QoS 等级、会话感知和持久连接，即使在困难的条件下也能保证消息的可靠传递，使其非常适合物联网应用。
- **安全通信**：安全对于物联网网络至关重要，因为其经常涉及敏感数据的传输。为确保数据在传输过程中的机密性，MQTT 提供传输层安全 (TLS) 和安全套接层 (SSL) 加密功能。此外，MQTT 还通过用户名/密码凭证或客户端证书提供身份验证和授权机制，以保护网络及其资源的访问。
- **双向通信**：MQTT 的发布-订阅模式为设备之间提供了无缝的双向通信方式。客户端既可以向主题发布消息，也可以订阅接收特定主题上的消息，从而实现了物联网生态系统中的高效数据交换，而无需直接将设备耦合在一起。这种模式也简化了新设备的集成，同时保证了系统易于扩展。
- **连续、有状态的会话**：MQTT 提供了客户端与 Broker 之间保持有状态会话的能力，这使得系统即使在断开连接后也能记住订阅和未传递的消息。此外，客户端还可以在建立连接时指定一个保活间隔，这会促使 Broker 定期检查连接状态。如果连接中断，Broker 会储存未传递的

消息（根据 QoS 级别确定），并在客户端重新连接时尝试传递它们。这个特性保证了通信的可靠性，降低了因间断性连接而导致数据丢失的风险。

- **大规模物联网设备支持：**物联网系统往往涉及大量设备，需要一种能够处理大规模部署的协议。MQTT 的轻量级特性、低带宽消耗和对资源的高效利用使其成为大规模物联网应用的理想选择。通过采用发布-订阅模式，MQTT 实现了发送者和接收者的解耦，从而有效地减少了网络流量和资源使用。此外，协议对不同 QoS 等级的支持使得消息传递可以根据需求进行定制，确保在各种场景下获得最佳的性能表现。
- **语言支持：**物联网系统包含使用各种编程语言开发的设备 and 应用。MQTT 具有广泛的语言支持，使其能够轻松与多个平台和技术进行集成，从而实现了物联网生态系统中的无缝通信和互操作性。您可以阅读我们的 [MQTT 客户端编程](#) 系列文章，学习如何在 PHP、Node.js、Python、Golang、Node.js 等编程语言中使用 MQTT。

MQTT 的工作原理

MQTT 是基于发布-订阅模式的通信协议，由 MQTT 客户端通过主题（Topic）发布或订阅消息，通过 MQTT Broker 集中管理消息路由，并依据预设的服务质量等级(QoS)确保端到端消息传递可靠性。

MQTT 客户端

任何运行 [MQTT 客户端库](#) 的应用或设备都是 MQTT 客户端。例如，使用 MQTT 的即时通讯应用是客户端，使用 MQTT 上报数据的各种传感器是客户端，各种 [MQTT 测试工具](#) 也是客户端。

MQTT Broker

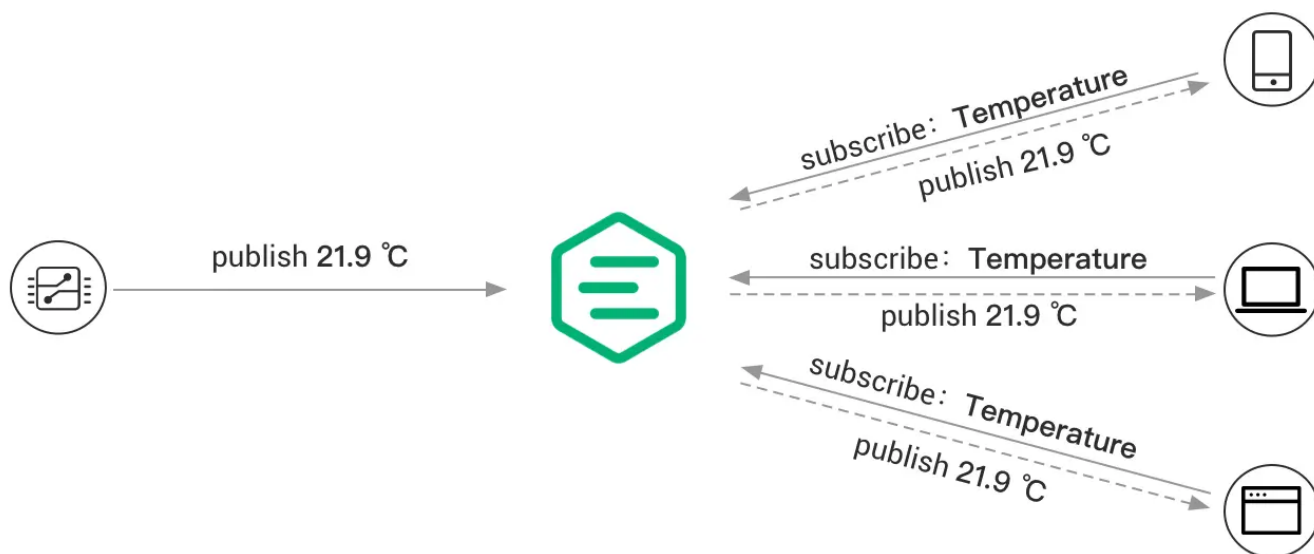
MQTT Broker 是负责处理客户端请求的关键组件，包括建立连接、断开连接、订阅和取消订阅等操作，同时还负责消息的转发。一个高效强大的 MQTT Broker 能够轻松应对海量连接和百万级消息吞吐量，从而帮助物联网服务提供商专注于业务发展，快速构建可靠的 MQTT 应用。

关于 MQTT Broker 的更多详情，请参阅文章 [2025 年最全面的 MQTT Broker 比较指南](#)。

发布-订阅模式

发布-订阅模式与客户端-服务器模式的不同之处在于，它将发送消息的客户端（发布者）和接收消息的客户端（订阅者）进行了解耦。发布者和订阅者之间无需建立直接连接，而是通过 MQTT Broker 来负责消息的路由和分发。

下图展示了 MQTT 发布/订阅过程。温度传感器作为客户端连接到 MQTT Broker，并通过发布操作将温度数据发布到一个特定主题（例如 `Temperature`）。MQTT Broker 接收到该消息后会负责将其转发给订阅了相应主题（`Temperature`）的订阅者客户端。



主题

MQTT 协议根据主题来转发消息。主题通过 `/` 来区分层级，类似于 URL 路径，例如：

```
▼ Plain Text |
1 chat/room/1
2
3 sensor/10/temperature
4
5 sensor/+/temperature
```

MQTT 主题支持以下两种通配符：`+` 和 `#`。

- `+`：表示单层通配符，例如 `a/+` 匹配 `a/x` 或 `a/y`。
- `#`：表示多层通配符，例如 `a/#` 匹配 `a/x`、`a/b/c/d`。

注意：通配符主题只能用于订阅，不能用于发布。

关于 MQTT 主题的更多详情，请参阅文章[通过案例理解 MQTT 主题与通配符](#)。

QoS

MQTT 提供了三种服务质量（QoS），在不同网络环境下保证消息的可靠性。

- QoS 0：消息最多传送一次。如果当前客户端不可用，它将丢失这条消息。
- QoS 1：消息至少传送一次。
- QoS 2：消息只传送一次。

关于 MQTT QoS 的更多详情，请参阅文章 [MQTT QoS 0, 1, 2 介绍](#)。

MQTT 的工作流程

在了解了 MQTT 的基本组件之后，让我们来看看它的一般工作流程：

1. 客户端使用 TCP/IP 协议与 Broker 建立连接，可以选择使用 TLS/SSL 加密来实现安全通信。客户端提供认证信息，并指定会话类型（Clean Session 或 Persistent Session）。
2. 客户端既可以向特定主题发布消息，也可以订阅主题以接收消息。当客户端发布消息时，它会将消息发送给 MQTT Broker；而当客户端订阅消息时，它会接收与订阅主题相关的消息。
3. MQTT Broker 接收发布的消息，并将这些消息转发给订阅了对应主题的客户端。它根据 QoS 等级确保消息可靠传递，并根据会话类型为断开连接的客户端存储消息。

开始使用 MQTT：快速教程

下面我们将通过一些简单的示例来展示如何使用 MQTT。在开始之前，需要准备 MQTT Broker 和 MQTT 客户端。

准备 MQTT Broker

您可以选择私有部署或完全托管的云服务来建立自己的 MQTT Broker。或者您也可以使用免费的公共 Broker。

EMQX 是一个可以「无限连接、任意集成、随处运行」的大规模分布式企业级 MQTT 物联网接入平台。它根据用户的不同需求提供了多个版本选择：

- 全托管的云服务

通过全托管的云服务启动 MQTT 服务是最便捷的方式。如下图所示，EMQX Serverless 版本是基于多租户架构的 MQTT 云服务，具有按量付费和灵活扩容的特性，可以在几分钟内启动，并在 AWS、Google Cloud 和 Microsoft Azure 的 17 个区域提供运行支持。

免费试用 EMQX Serverless

无须绑定信用卡

[开始试用 →](#)

- 免费的公共 MQTT Broker

在本文中，我们将使用 EMQ 提供的[免费的公共 MQTT 服务器](#)，它基于完全托管的 [MQTT 云服务 – EMQX Cloud](#) 创建。服务器信息如下：

Server: `broker.emqx.io`

TCP Port: `1883`

WebSocket Port: `8083`

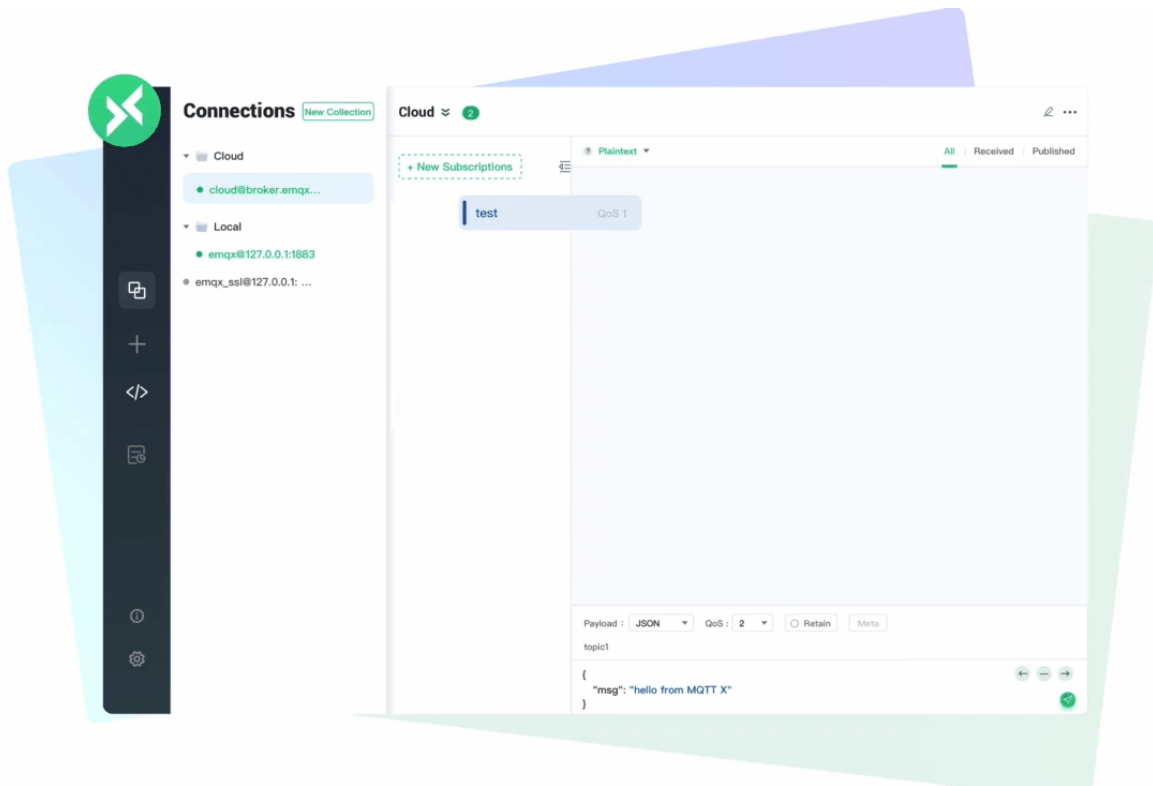
SSL/TLS Port: `8883`

Secure WebSocket Port: `8084`

准备 MQTT 客户端

在本文中，我们将使用 [MQTTX](#) 提供的支持浏览器访问的 MQTT 客户端工具，访问地址为 <https://mqttx.app/web-client/>。MQTTX 还提供了[桌面客户端](#)和[命令行工具](#)。

[MQTTX](#) 是一款跨平台的 MQTT 5.0 桌面客户端，可在 macOS、Linux、Windows 操作系统上运行。其用户友好的聊天式界面使用户能够轻松创建多个 MQTT/MQTTS 连接，并进行 MQTT 消息的订阅和发布。



MQTTX 界面

目前，各种编程语言都拥有成熟的开源 MQTT 客户端库。我们在[流行的 MQTT 客户端库和 SDK](#)中精选了多个编程语言的 MQTT 客户端库，并提供了详细的代码示例，旨在帮助您快速了解 MQTT 客户端的使用。

创建 MQTT 连接

在使用 MQTT 协议进行通信之前，客户端需要创建一个 MQTT 连接来连接到 Broker。

在浏览器中打开 <https://mqttx.app/web-client/>，点击页面中间的 **New Connection** 按钮，将看到如下页面。

< Back New Connect

General

* Name Simple Demo ⓘ

* Client ID mqttx_b2c892c7 ⓘ

* Host ws:// broker.emqx.io

* Port 8083

* Path /mqtt

Username

Password

SSL/TLS ☐ true ☒ false

Advanced ▲

Connect Timeout (s) 10

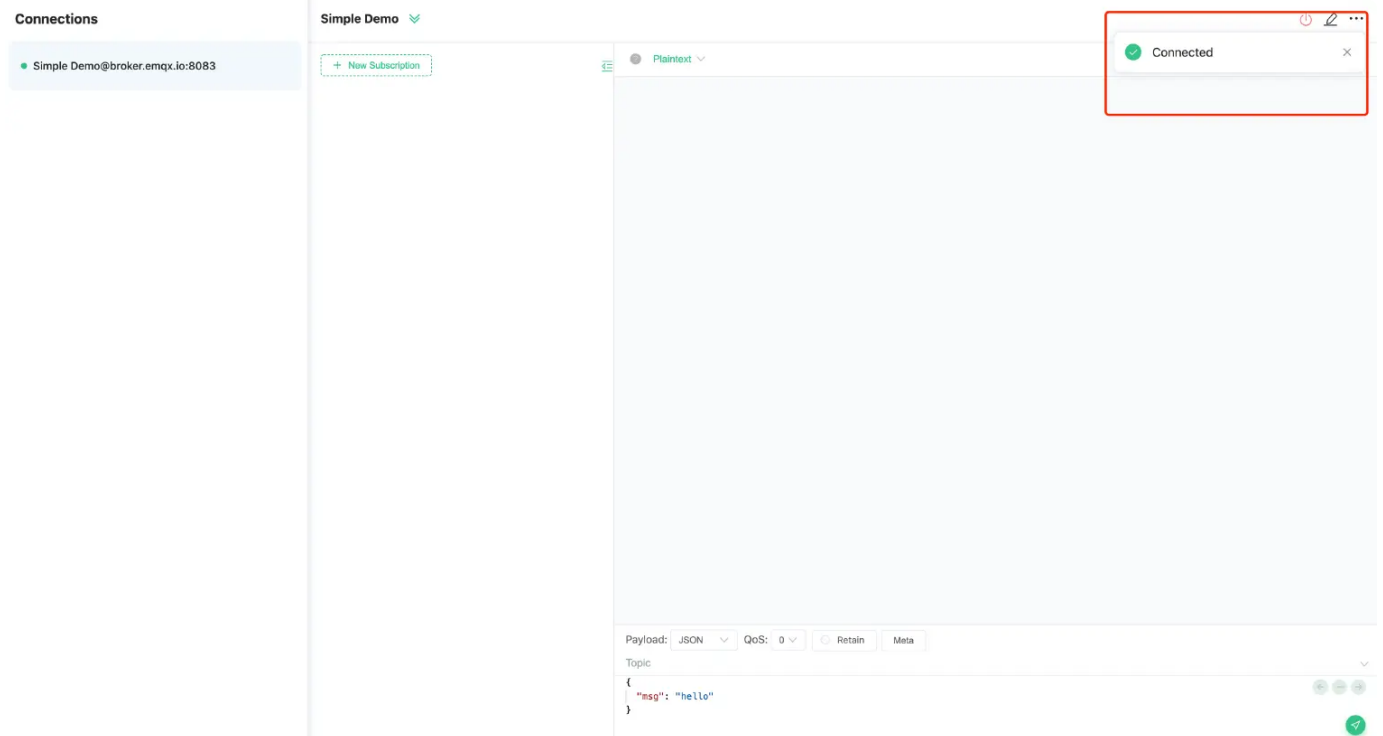
Keep Alive (s) 60

Clean Session ☒ true ☐ false

Auto Reconnect ☐ true ☒ false

MQTT Version 5.0 ▼

我们在 **Name** 中输入 **Simple Demo**，然后点击右上角的 **Connect** 按钮，建立一个 MQTT 连接。如下图所示，表示连接成功。

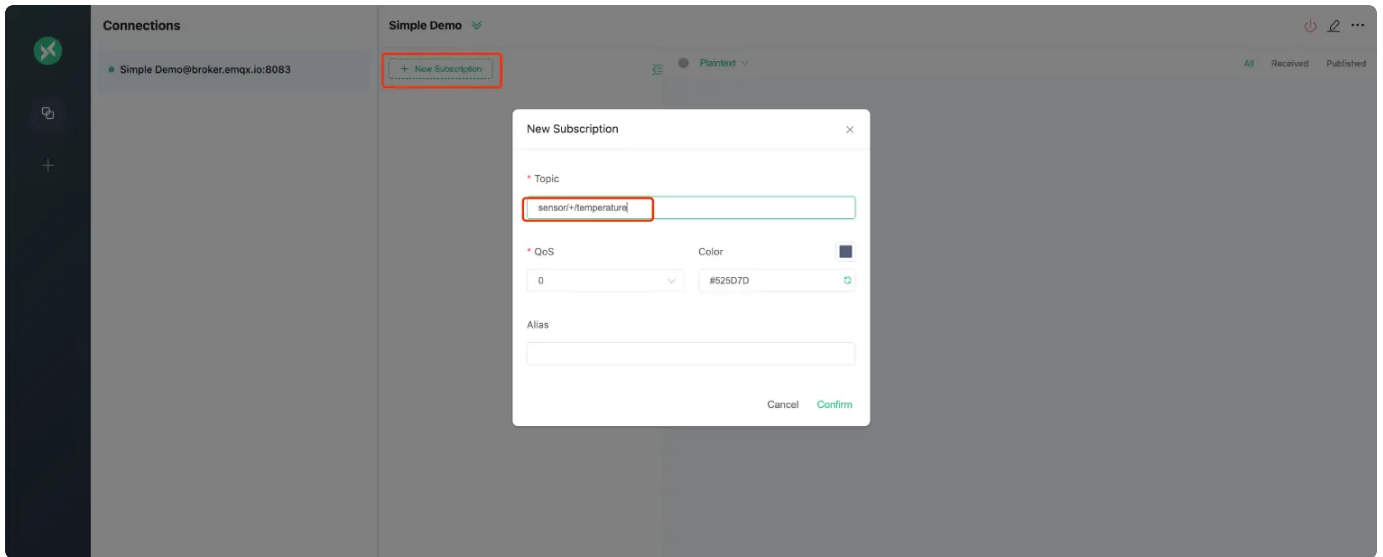


要了解更多关于 MQTT 连接参数的内容，请查看我们的文章：[建立 MQTT 连接时如何设置参数](#)。

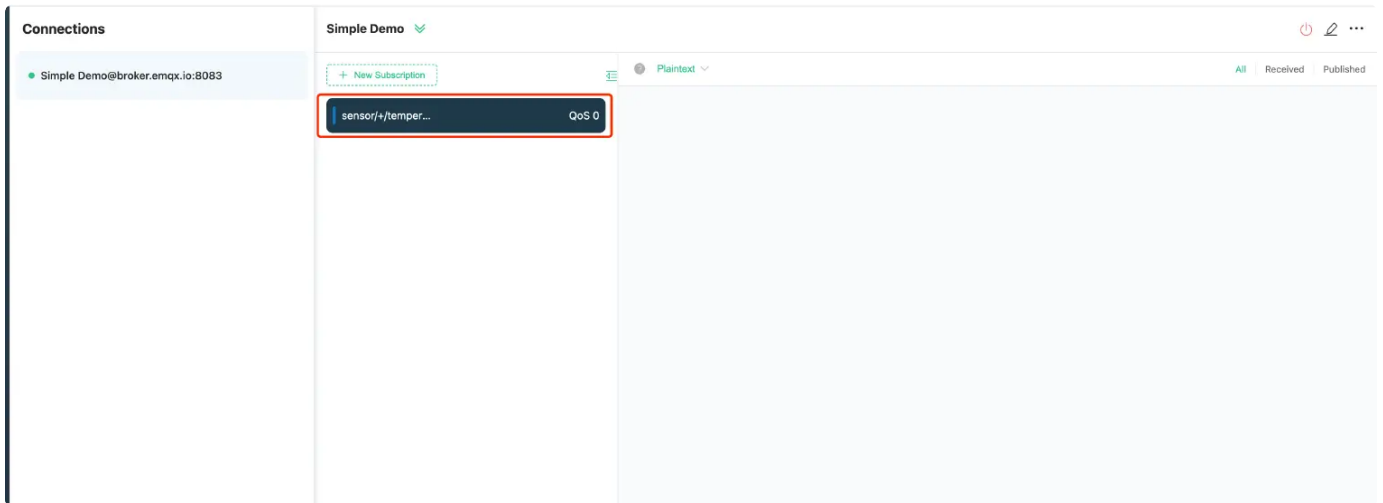
通过通配符订阅主题

接下来，我们在上面创建的 `Simple Demo` 连接中通过通配符订阅主题 `sensor/+/temperature`，这样就可以接收所有传感器发送的温度数据了。

如下图所示，点击 `New Subscription` 按钮，在弹出框中的 `Topic` 字段中输入主题 `sensor/+/temperature`，QoS 保持默认值 0。

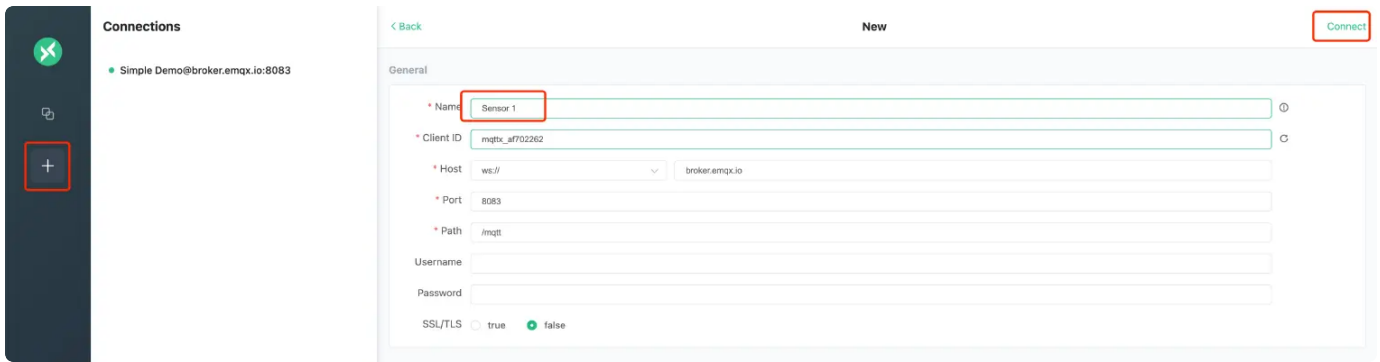


订阅成功后，会在订阅列表的中间看到新增了一条记录。

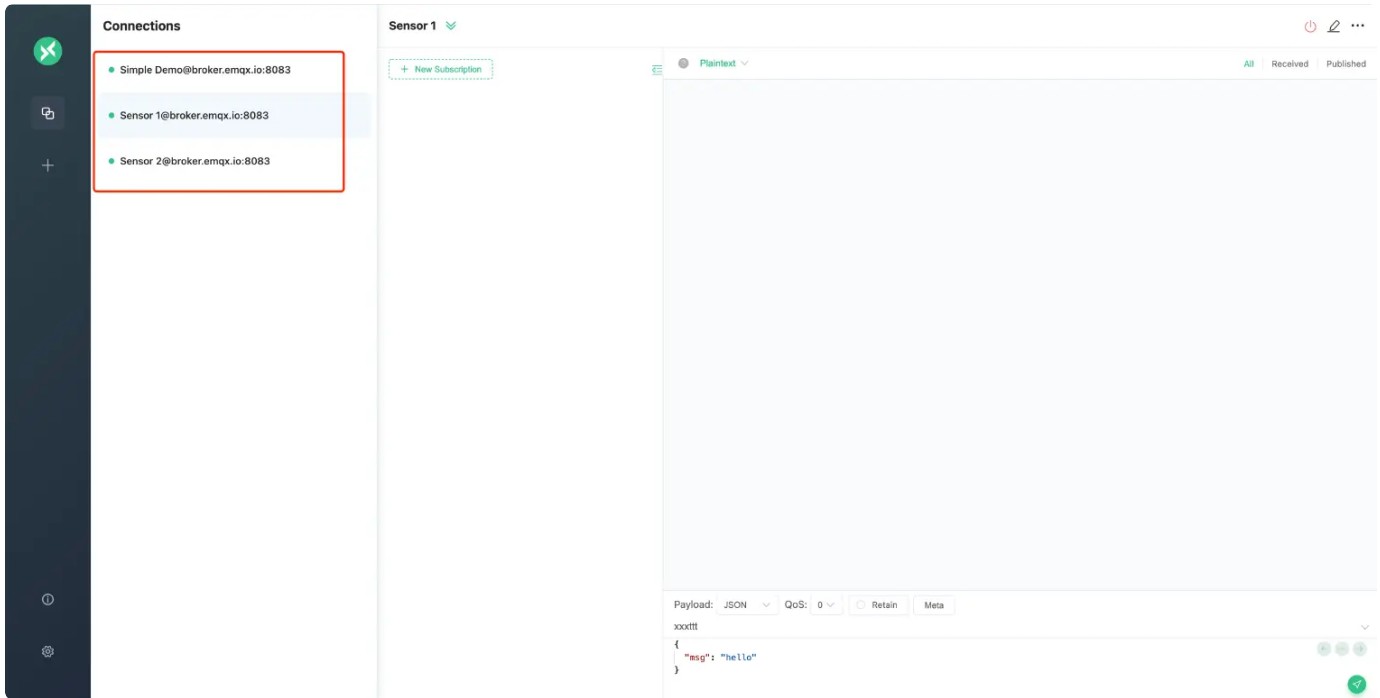


发布 MQTT 消息

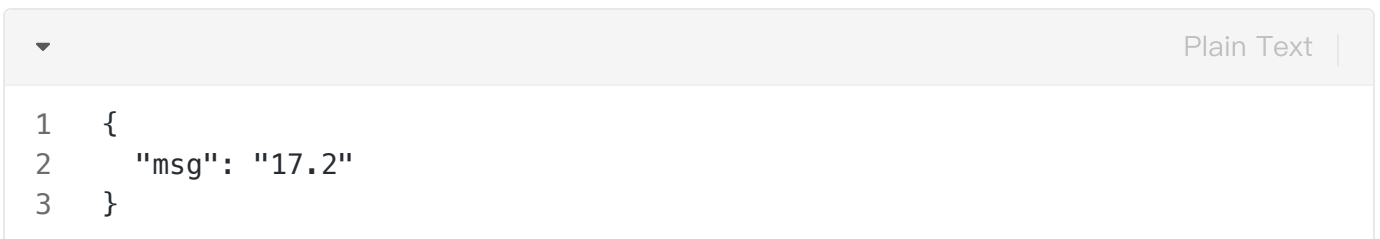
接下来，我们点击左侧菜单上的 `+` 按钮创建两个连接，分别命名为 `Sensor 1` 和 `Sensor 2`，用来模拟两个温度传感器。

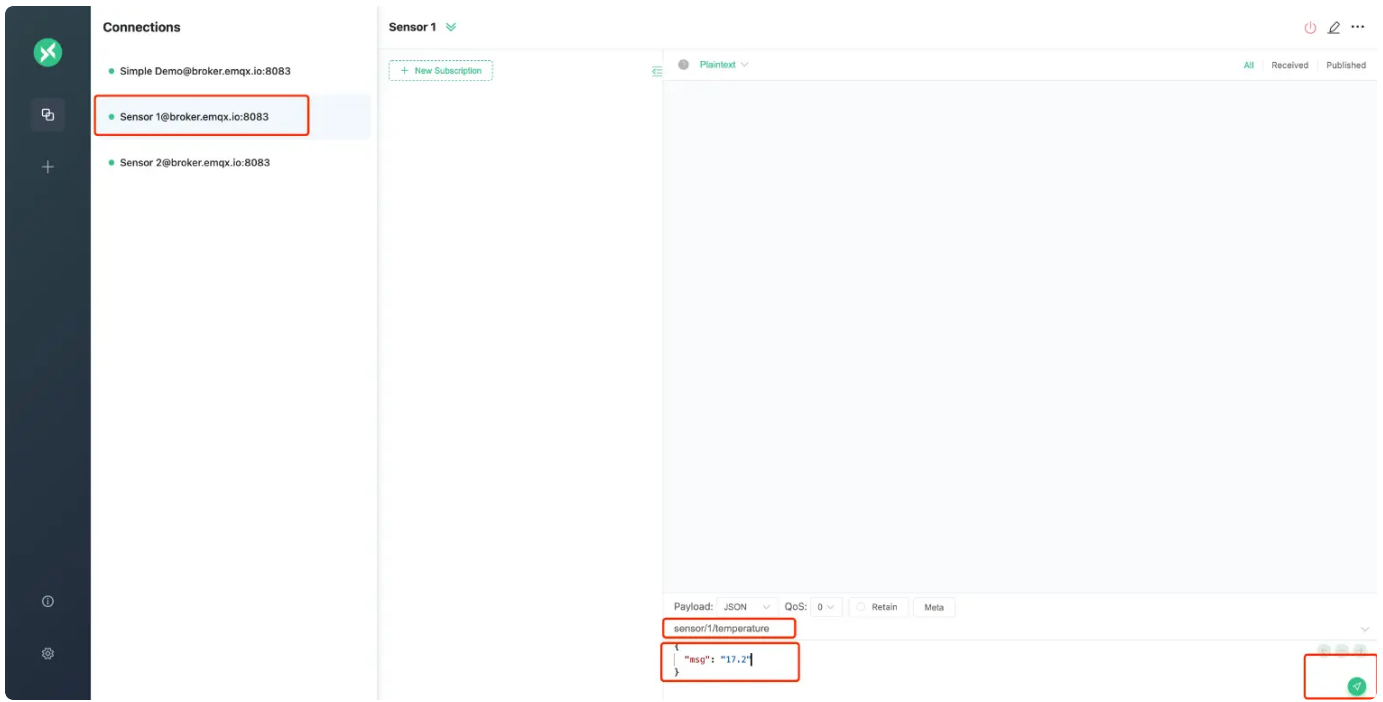


连接创建成功后，会看到三个连接，每个连接左侧的在线状态指示灯都是绿色的。

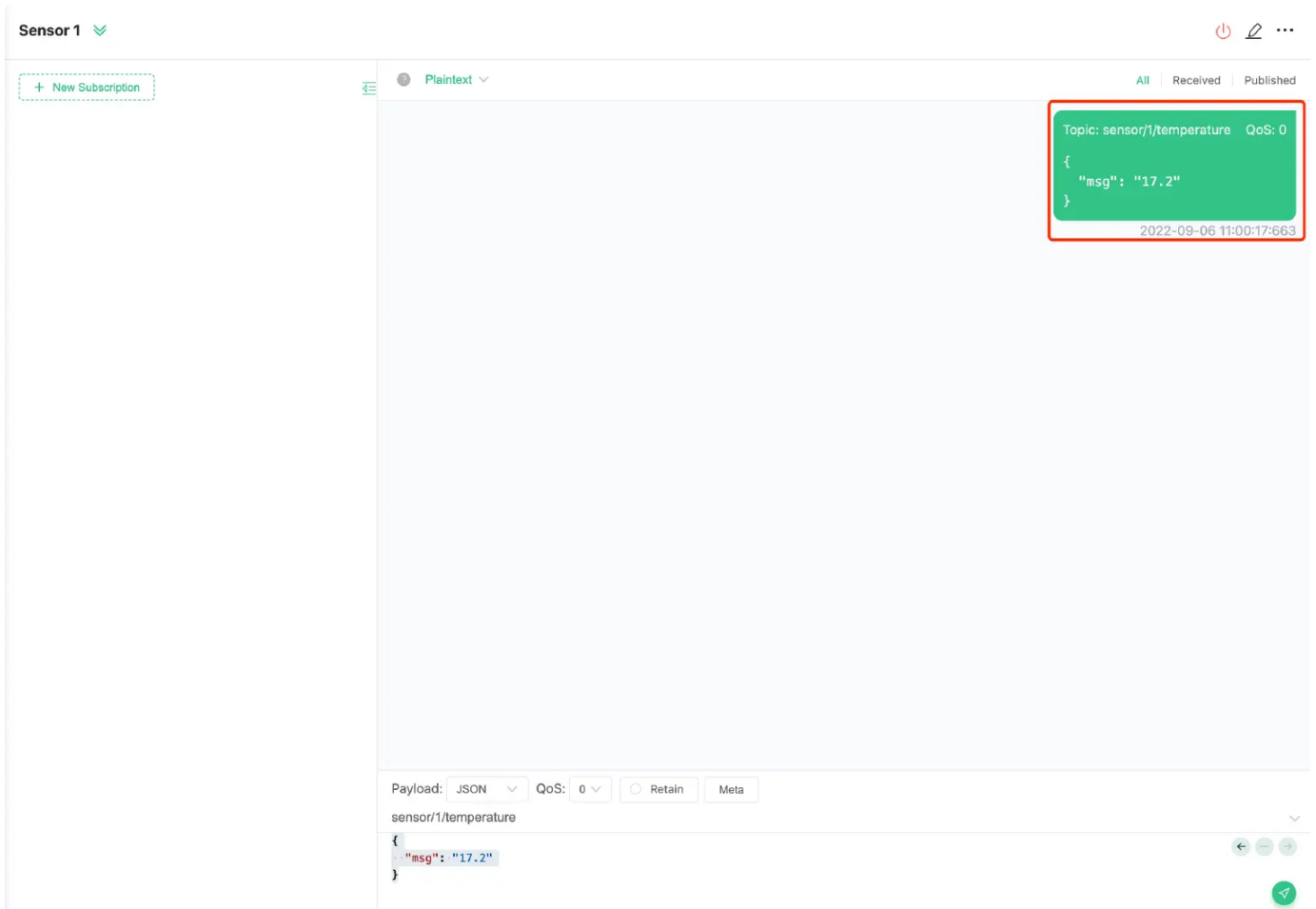


选择 **Sensor 1** 连接，在页面下方的发布主题中输入 **sensor/1/temperature**，在消息框中输入以下 JSON 格式的消息，然后点击右下方的发布按钮发送消息。





如下图所示，消息发送成功。

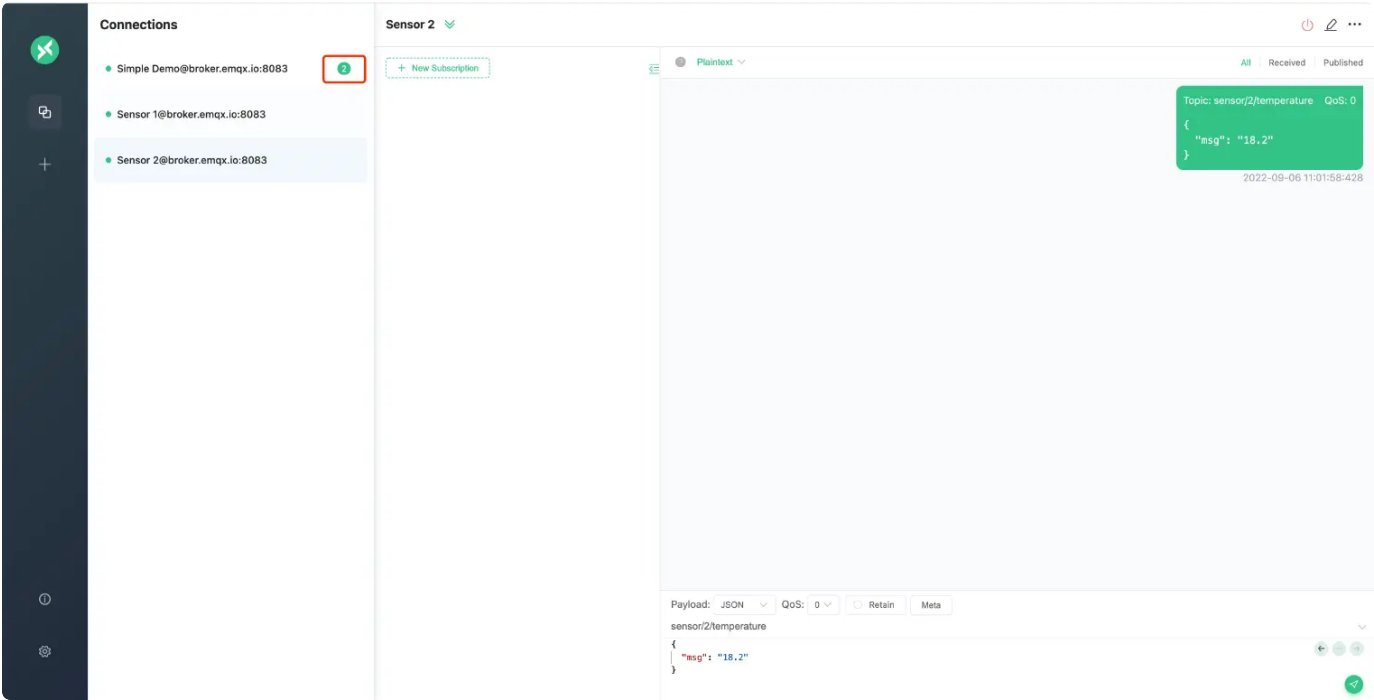


使用相同的步骤，在 **Sensor 2** 连接中发布以下 JSON 消息到 **sensor/2/temperature** 主题。

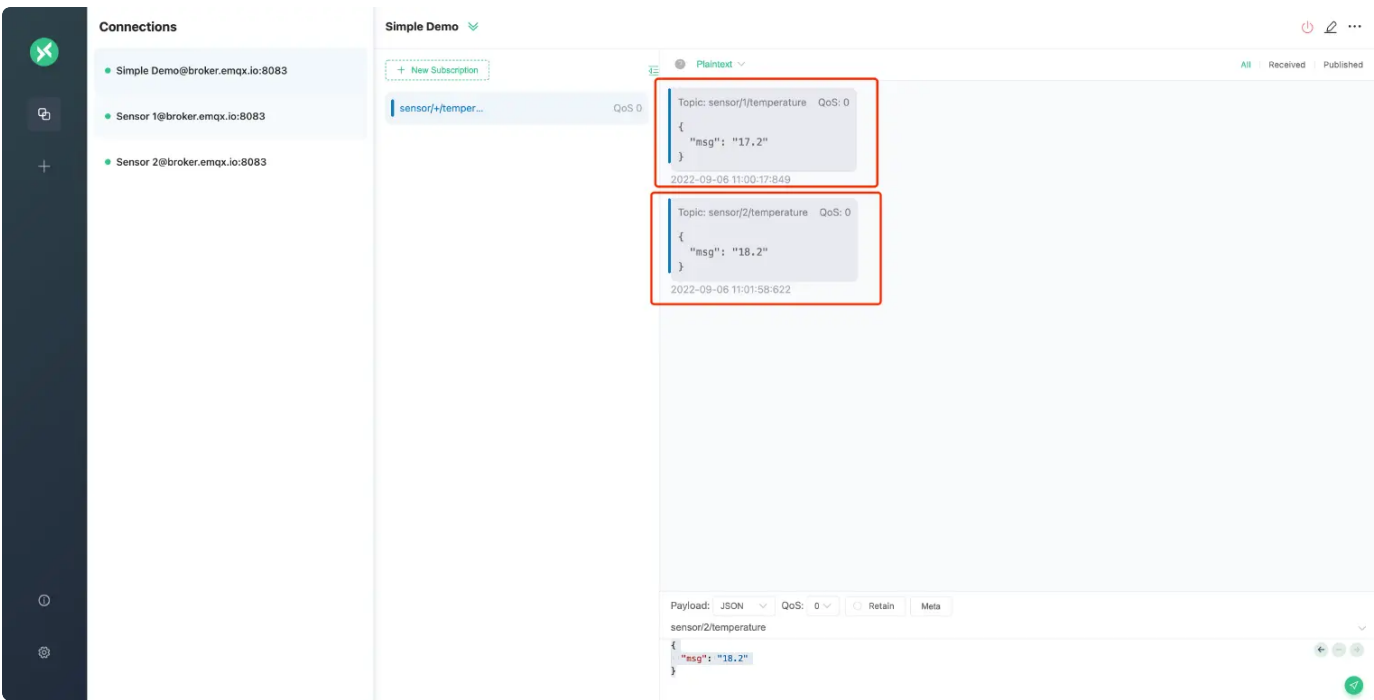
Plain Text

```
1  {
2    "msg": "18.2"
3  }
```

您会看到 **Simple Demo** 连接收到两条新消息。



点击 **Simple Demo** 连接，会看到两个传感器发送的两条消息。

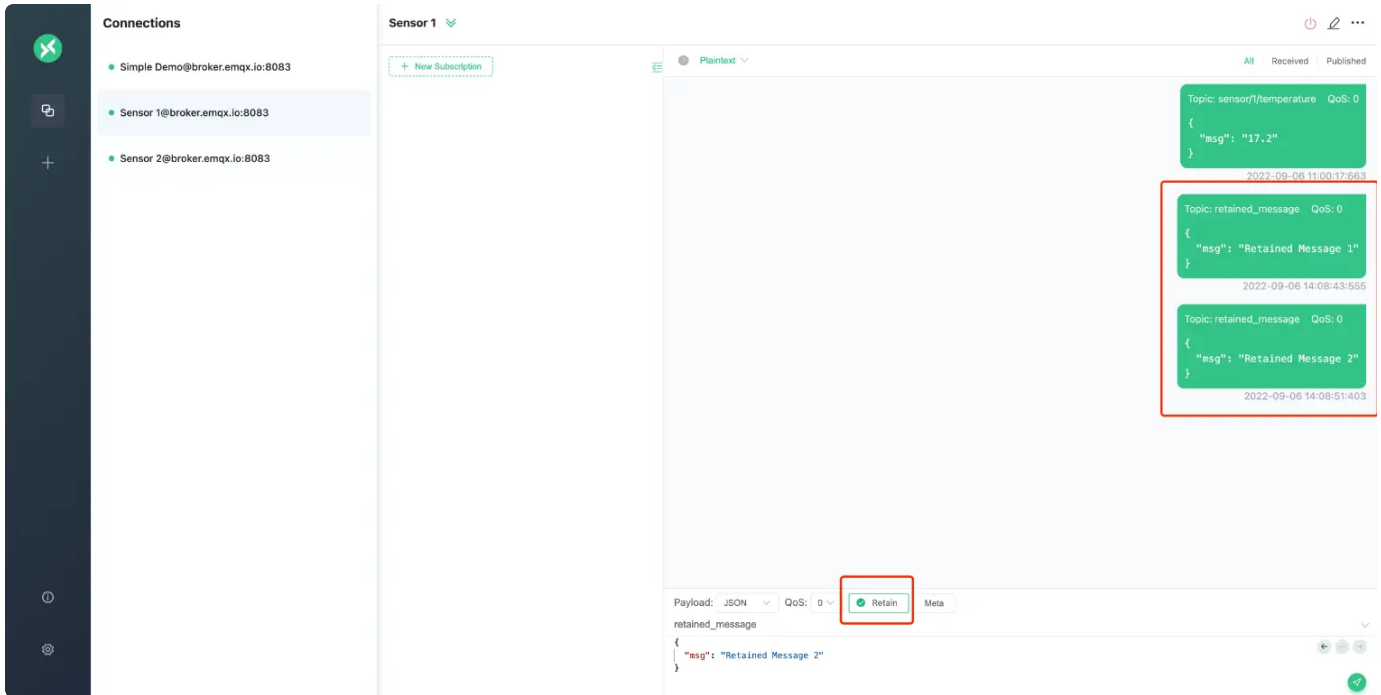


MQTT 功能演示

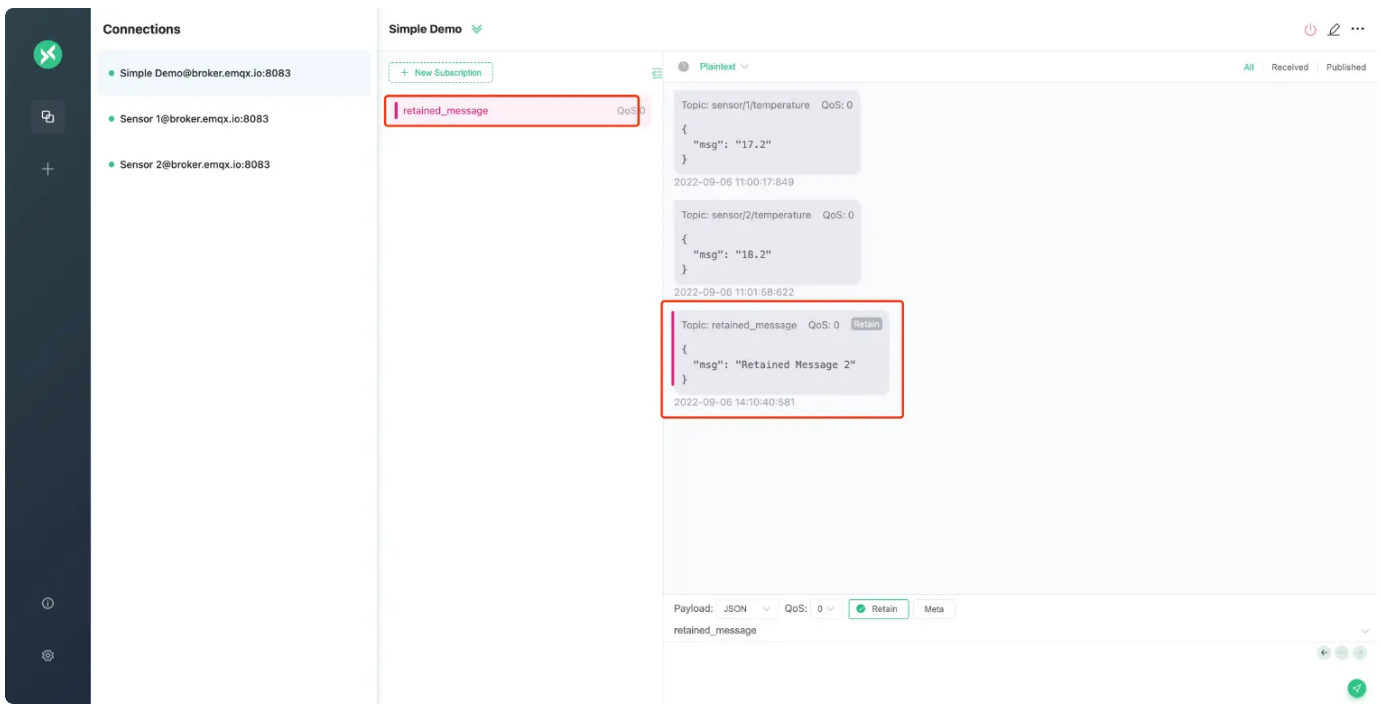
保留消息

当 MQTT 客户端向服务器发布消息时，可以设置保留消息标志。保留消息存储在消息服务器上，后续订阅该主题的客户端仍然可以收到该消息。

如下图所示，我们在 **Sensor 1** 连接中勾选 **Retain** 选项，然后向 **retained_message** 主题发送两条消息。



接着，我们在 **Simple Demo** 连接中订阅 **retained_message** 主题。订阅成功后，会收到 **Sensor 1** 发送的第二条保留消息，这说明服务器只会为主题保留最近的一条保留消息。



关于保留消息的更多细节，请阅读文章 [MQTT 保留消息初学者指南](#)。

Clean Session

MQTT 客户端通常只能在在线状态下接收其它客户端发布的消息。如果客户端离线后重新上线，它将无法收到离线期间的消息。

但是，如果客户端连接时设置 Clean Session 为 false，并且使用相同的客户端 ID 再次上线，那么消息服务器将为客户端缓存一定数量的离线消息，并在它重新上线时发送给它。

本次演示使用的公共 MQTT 服务器设置为缓存 5 分钟的离线消息，最大消息数为 1000 条，且不保存 QoS 0 消息。

下面，我们创建一个 MQTT 3.1.1 连接，并用 QoS 1 来演示 Clean Session 的使用。

MQTT 5.0 中将 Clean Session 拆分成了 Clean Start 与 Session Expiry Interval。详情请参考文章 [Clean Start 与 Session Expiry Interval](#)。

创建一个名为 MQTT V3 的连接，设置 Clean Session 为 false，选择 MQTT 版本为 3.1.1。

Connections

- Simple Demo@broker.emqx.io:8083
- Sensor 1@broker.emqx.io:8083
- Sensor 2@broker.emqx.io:8083

New Connect

General

* Name

* Client ID

* Host

* Port

* Path

Username

Password

SSL/TLS ☐ true ☒ false

Advanced

Connect Timeout (s)

Keep Alive (s)

Clean Session ☐ true ☒ false

Auto Reconnect ☐ true ☒ false

MQTT Version

Last Will and Testament

Last-Will Topic

Last-Will QoS ☒ 0 ☐ 1 ☐ 2

Last-Will Retain ☐ true ☒ false

连接成功后，订阅 `clean_session_false` 主题，并将 QoS 设置为 1。

Connections

- Simple Demo@broker.emqx.io:8083
- Sensor 1@broker.emqx.io:8083
- Sensor 2@broker.emqx.io:8083
- MQTT_V3@broker.emqx.io:8083

MQTT_V3 Disconnect Plaintext All Received Published

New Subscription

* Topic

* QoS

Color

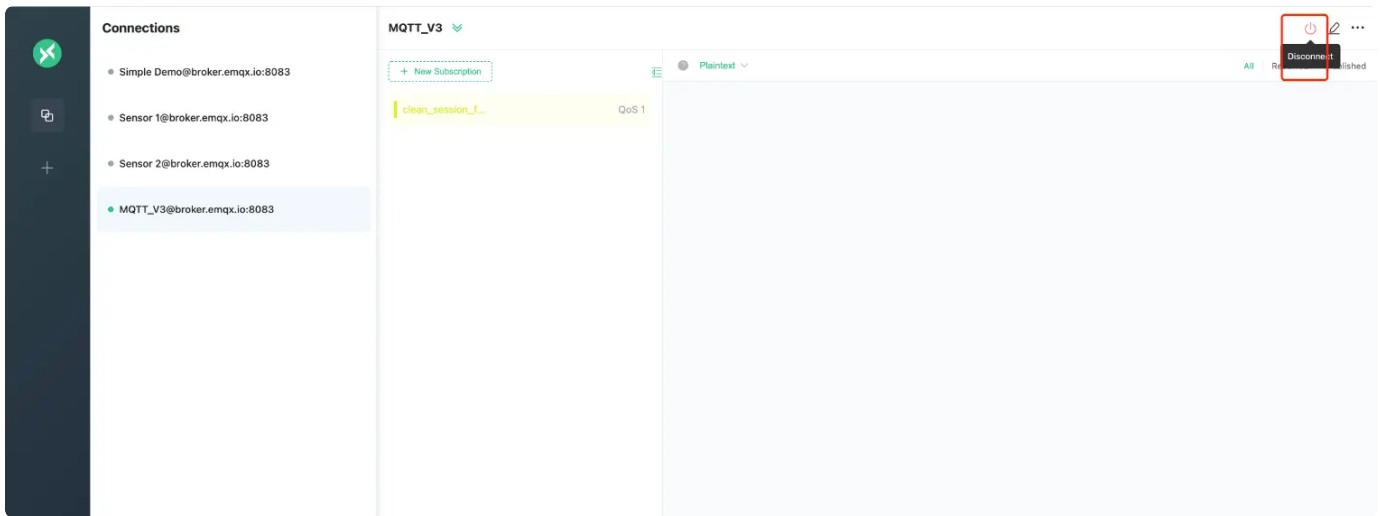
Alias

Cancel Confirm

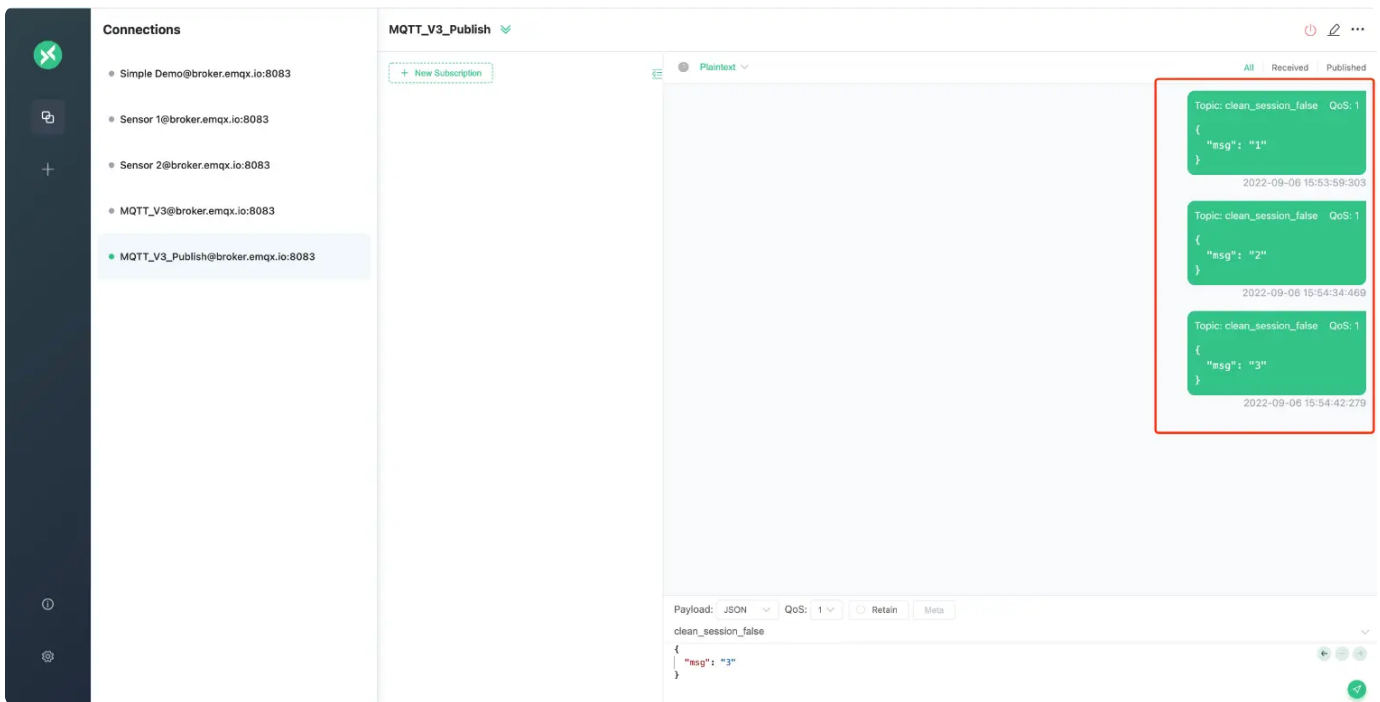
Payload: JSON QoS: 1 Retain Mode

```
testtopic1
{
  "msg": "3"
}
```

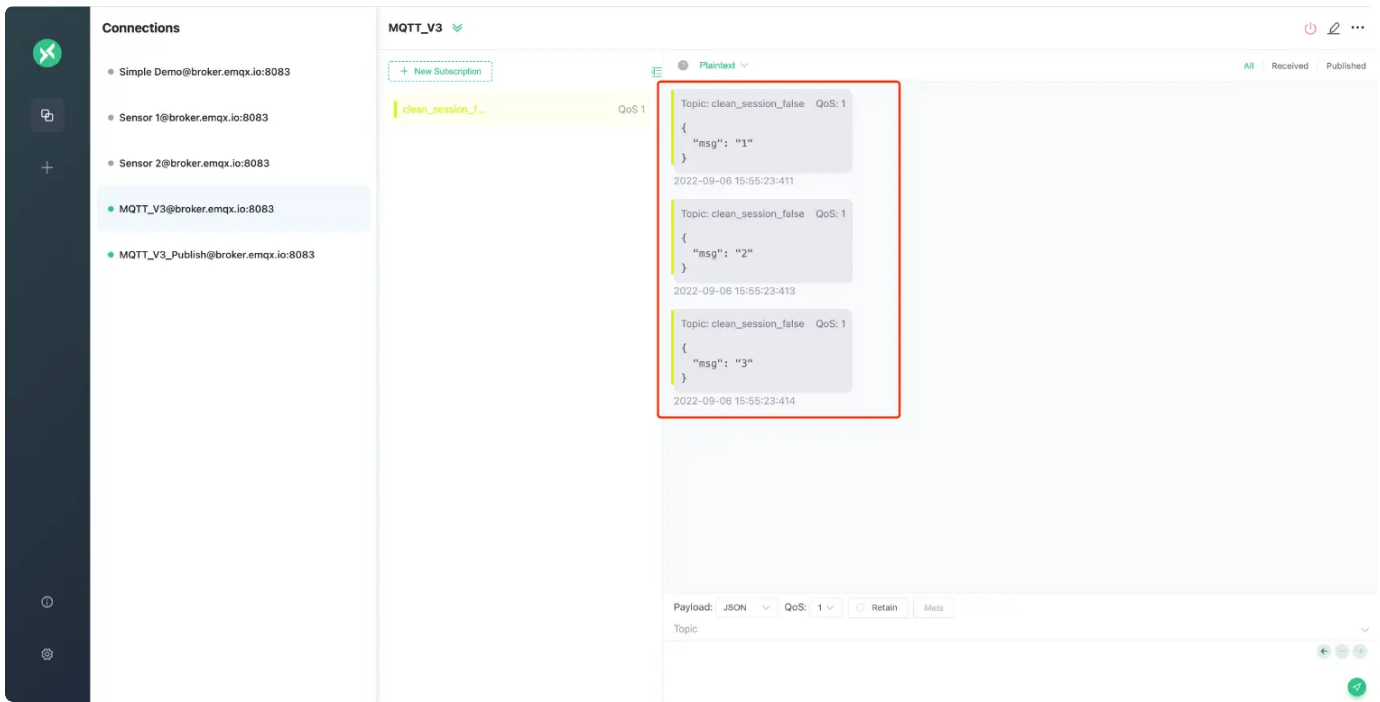
订阅成功后，点击右上角的断开按钮，断开连接。



然后，创建一个名为 `MQTT_V3_Publish` 的连接，MQTT 版本也设置为 3.1.1。连接成功后，向 `clean_session_false` 主题发布三条消息。



接着，选择 `MQTT_V3` 连接，点击连接按钮重新连接到服务器，会收到三条离线消息。



关于 Clean Session 的更多细节，请阅读文章 [MQTT Persistent Session 与 Clean Session 详解](#)。

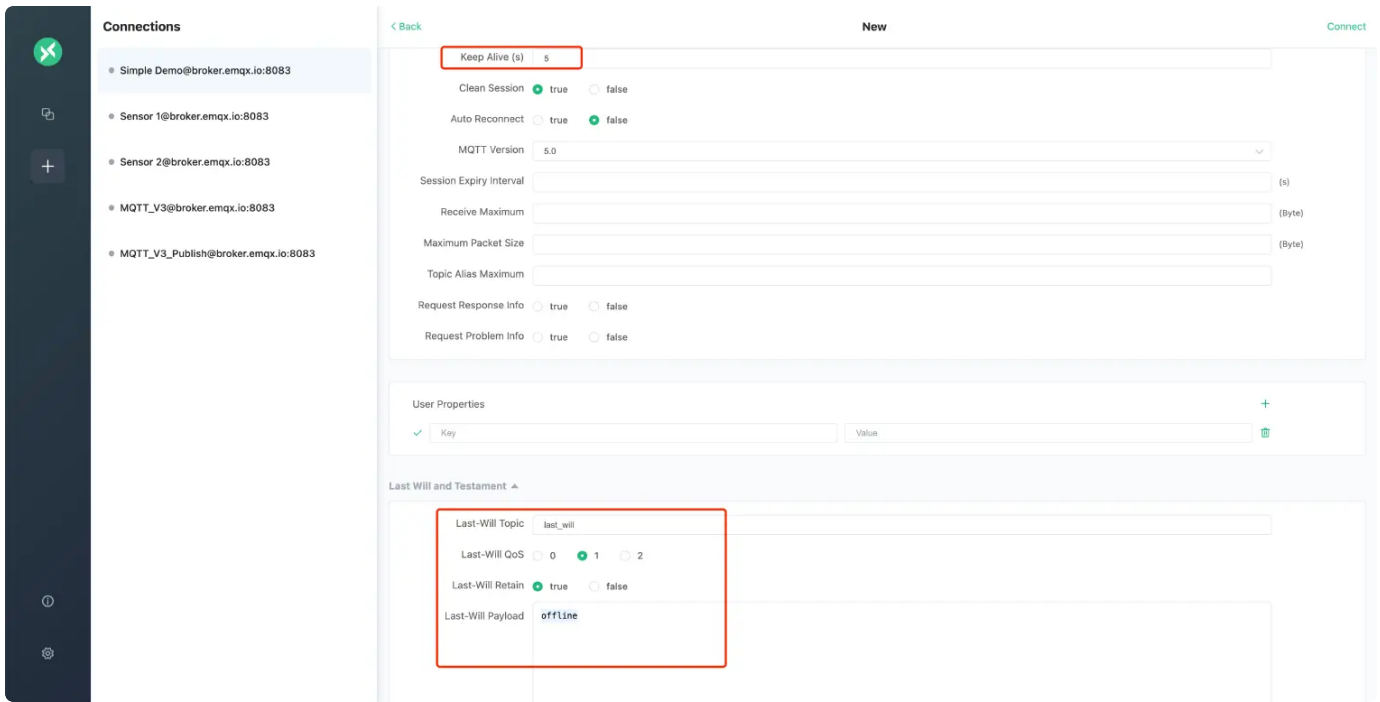
遗嘱消息

MQTT 客户端在向服务器发起 CONNECT 请求时，可以选择是否发送遗嘱消息标志，并指定遗嘱消息的主题和有效载荷。

如果 MQTT 客户端异常离线（在断开连接前没有向服务器发送 DISCONNECT 消息），MQTT 服务器会发布遗嘱消息。

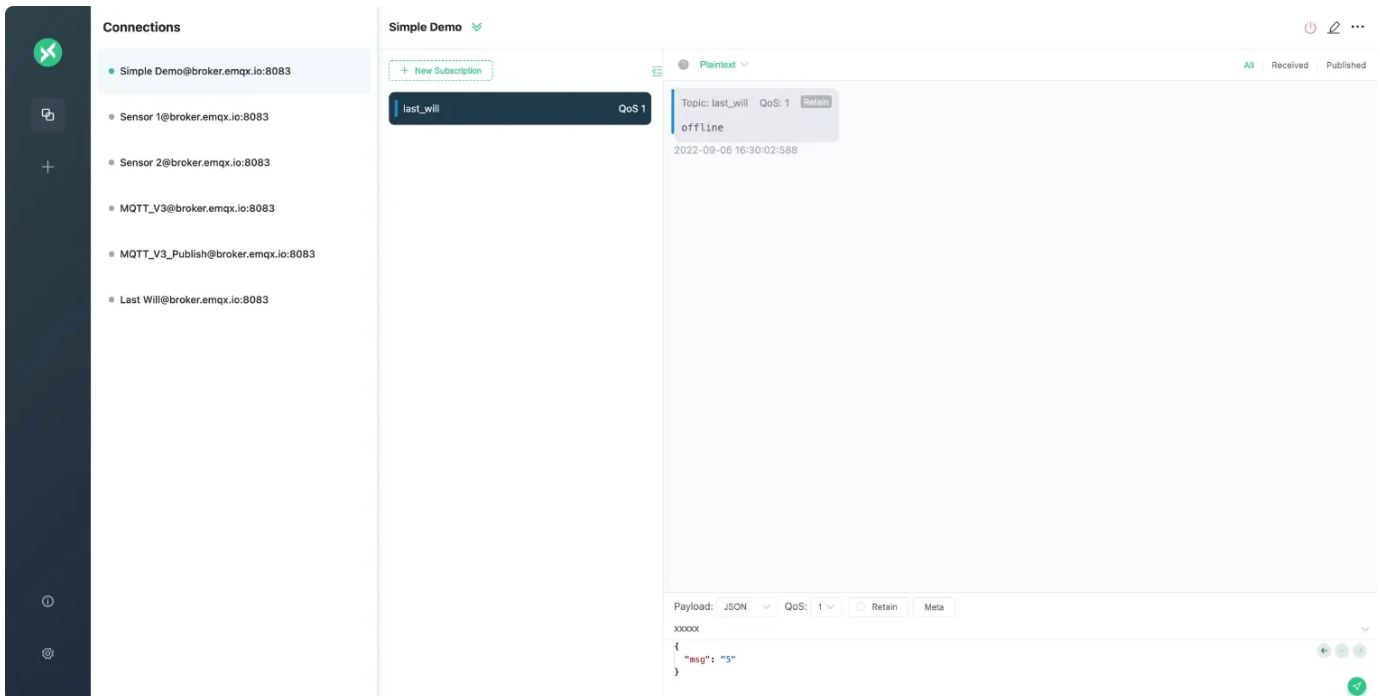
我们创建一个名为 `Last Will` 的连接来演示这个功能。

- 为了快速看到效果，我们把 Keep Alive 设置为 5 秒。
- Last-Will Topic 设置为 `last_will`。
- Last-Will QoS 设置为 `1`。
- Last-Will Retain 设置为 `true`。
- Last-Will Payload 设置为 `offline`。



连接成功后，我们断开电脑网络超过 5 秒（模拟客户端异常断开连接），然后再恢复网络。

接着启动 Simple Demo 连接，并订阅 `last_will` 主题。您会收到 `Last Will` 连接设置的遗嘱消息。



关于 MQTT 遗嘱消息的更多内容，请阅读文章 [MQTT 遗嘱消息的使用](#)。

协议对比：MQTT 的核心优势

除 MQTT 外，HTTP、WebSocket 和 CoAP 等协议也常用于物联网领域。相较而言，MQTT 通过异步通信机制展现出独特优势：更低的带宽占用、轻量级发布-订阅模型，以及优化的报文头设计。这些使其特别适用于资源受限环境和大规模设备网络部署场景。

MQTT 与其他协议的深度对比分析可参考以下专题文章：

- [MQTT vs HTTP](#)
- [MQTT vs WebSocket](#)
- [MQTT vs CoAP](#)
- [MQTT vs AMQP](#)

MQTT 进阶

MQTT 安全最佳实践

MQTT 安全性在物联网应用中至关重要——物联网设备通常处理敏感数据，若缺乏完善的安全防护，攻击者可能利用漏洞拦截消息、篡改数据或破坏关键系统，甚至造成严重损害。为确保 MQTT 的安全性，通常采用以下几种方法：

- 认证
 - [用户名/密码认证](#)
 - [SCRAM 增强认证](#)
 - [其他认证方法](#)
- [授权](#)
- [流量控制](#)
- [TLS/SSL](#)

通过采取这些防护措施，企业能够有效保障 MQTT 通信安全，维护物联网系统的完整性与机密性，构建可靠的安全防护体系。

MQTT 数据存储

通过 MQTT 连接的数百万设备持续产生有价值的数据，当这些数据被存储和分析时，其价值将得到进一步释放。但根据 MQTT 协议规范，MQTT Broker 本身并不具备数据存储功能。因此，必须将其与适当的数据库解决方案集成，才能有效管理和利用这些数据。选择合适的数据库不仅能优化数据存储效率，还能提升物联网应用的扩展性。

获取 MQTT 数据库选型指南：[Database for MQTT Data Storage: A Selection Guide.](#)

2025 年 MQTT 的 8 大趋势

MQTT over QUIC

QUIC 是 Google 推出的一种基于 UDP 的新型传输协议，能够降低延迟、提高数据传输速率。将 QUIC 引入 MQTT 将为网络不稳定或低延迟要求的场景（如车联网和工业物联网）带来优势。EMQX 和未来的 MQTT 版本正逐步采用 MQTT over QUIC，将引领物联网连接标准的重要变革。

更多详情，请查看博客：[MQTT over QUIC：物联网消息传输还有更多可能](#)

MQTT Serverless

Serverless MQTT 作为前沿的架构创新，使 MQTT 服务的快速部署变得轻而易举。此外，其资源自动扩展和按需付费模式提供了极大的灵活性，有望推动 MQTT 更广泛的应用，降低运营成本，激发不同行业的创新协作。我们甚至可能看到每个物联网和工业物联网开发者都能拥有一个免费的 Serverless MQTT 消息服务器。

免费试用 EMQX Serverless

无须绑定信用卡

[开始试用 →](#)

MQTT 多租户架构

多租户架构是实现 Serverless MQTT 服务的一个重要基础。不同用户或租户的物联网设备可以连接至同一大规模 MQTT 集群，同时保持各自的数据和业务逻辑独立。支持多租户的 MQTT 服务器将降低管理负担，提升复杂场景或大规模物联网应用的灵活性。

更多详情，请查看博客：[MQTT 服务新趋势：了解 MQTT 多租户架构](#)

MQTT Sparkplug 3.0

MQTT Sparkplug 定义了如何通过 MQTT 连接传感器、执行器、PLC 和网关等工业设备，旨在简化工业设备的连接与通信，实现高效的数据采集、处理和分析。最新的 3.0 版本引入了更多高

级功能，有望在工业物联网中得到更广泛的应用。

更多详情，请查看博客：[MQTT Sparkplug：在工业 4.0 时代架起 IT 和 OT 的桥梁](#)

MQTT 统一命名空间

统一命名空间（Unified Namespace）是一个建立在面向工业物联网和工业 4.0 的 MQTT Broker 上的解决方案架构。它采用星型拓扑，通过中央 MQTT Broker 连接工业设备、传感器和应用（如 SCADA、MES 和 ERP）。采用统一命名空间可以更高效地实现 OT 和 IT 系统的数据交换，最终实现统一。

更多详情，请查看博客：[统一命名空间（UNS）：面向工业物联网的下一代数据架构](#)

MQTT 跨域集群

MQTT 跨域集群是一种创新架构，允许部署在不同区域或云端的 MQTT Broker 协同工作，形成一个统一的集群。它支持在多云环境中构建全球 MQTT 访问网络，使得本地接入的设备和应用无论物理位置如何都能相互通信。

更多详情，请查看博客：[EMQX 跨域集群：增强可扩展性，打破地域限制](#)

MQTT Streams

MQTT Streams 是 MQTT 协议备受期待的一项扩展能力，能够在 MQTT Broker 内实时处理海量、高频的数据流。这一创新功能支持历史消息重播，确保数据一致性、审计和合规。内置的流处理功能将简化物联网数据处理架构，成为基于 MQTT 的物联网应用中实时数据管理的宝贵工具。

MQTT + AI

物联网的快速发展和人工智能的兴起为智能互联系统开辟了新可能。MQTT 正在连接物理设备世界与人工智能数字智慧，为 AI 应用构建"神经系统"——可靠而迅捷地传输信号——使大语言模型（LLM）等 AI 系统能够在互联环境中感知、推理并采取行动。这场革新才刚刚开始，但 MQTT 经过验证的技术特性与持续演进，意味着它将继续在未来数年成为 AIoT 创新的核心基础设施。

更多详情，请查看白皮书：[MQTT + AI 白皮书：数据赋能技术革命](#)

深入学习 MQTT

本文详细介绍了 MQTT 的基本概念和使用流程，您可以按照本文所学的内容尝试使用 MQTT 协议。